

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики

Отчёт

Дисциплина: Технология создания и поддержки ПО

Работу выполнила:
студентка 37 группы
Кузнецова А.А.

Краснодар
2024

Цель: представить текст ТЗ фитнес-приложения

Задачи: расписать подробно каждый пункт ТЗ и ответить на контрольные вопросы

Ход работы:

1. Цели и задачи проекта

Цель: создание удобной, функциональной и надежной платформы для пользователей, которая позволяет отслеживать физическую активность, состояние здоровья и достигать поставленных целей в фитнесе.

Задачи: обеспечить пользователя инструментами для мониторинга здоровья и тренировок; предоставить возможность использования готовых программ тренировок; предоставить возможность интеграции с внешними устройствами (смарт-часы); обеспечить удобным интерфейсом и социальными функциями (публикация результатов, получение напоминаний); администрировать контент и управлять пользователями.

2. Функциональные требования

2.1 Отслеживание показателей здоровья

- **F01.01:** Реализация функции отслеживания калорий и БЖУ.
 - **Требование:** Пользователь должен иметь возможность вводить и отслеживать данные о питании, включая калорийность и соотношение белков, жиров и углеводов.
- **F01.02:** Реализация функции мониторинга сна.
 - **Требование:** Приложение должно принимать данные о длительности и качестве сна (вручную или через синхронизацию с устройствами).
- **F01.03:** Реализация функции контроля водного баланса.

- Требование: Пользователь должен иметь возможность вводить данные о потреблении воды и отслеживать выполнение цели.

2.2 Отслеживание тренировок

- **F02.01:** Отслеживание силовых тренировок.
 - Требование: Приложение должно записывать параметры силовых тренировок (упражнение, количество подходов, повторений, вес).
- **F02.02:** Отслеживание кардио тренировок.
 - Требование: Пользователь может записывать данные о дистанции, времени, средней скорости и пульсе (при наличии устройства).

2.3 Установка целей

- **F03.01:** Установка целей по шагам.
 - Требование: Пользователь может задать цель (например, 10 000 шагов в день) и отслеживать её выполнение.

2.4 Социальные функции

- **U01.01:** Возможность делиться результатами тренировок в социальных сетях.
 - Требование: Пользователь может опубликовать результаты тренировок на выбранной платформе (Telegram, Instagram и др.).

2.5 Готовые тренировки

- **F04.01:** Пользователь должен иметь доступ к базе готовых тренировочных планов.
 - Требование: Приложение предоставляет рекомендации по тренировкам в зависимости от целей (похудение, набор массы).

2.6 Синхронизация устройств

- **F05.01:** Синхронизация со смарт-часами.

- Требование: Данные (шаги, пульс, тренировки) должны синхронизироваться через API смарт-часов.

3. Нефункциональные требования

3.1 Производительность

- **R01.01:** Приложение должно поддерживать до 10 000 активных пользователей одновременно.

3.2 Надежность

- **R02.01:** Гарантировать сохранность пользовательских данных при сбоях системы.

3.3 Удобство использования

- **U02.01:** Интерфейс должен быть адаптирован для iOS и Android.
- **U02.02:** Среднее время выполнения основной функции (например, добавление тренировки) не должно превышать 2 секунд.

3.4 Информационная безопасность

- **S01.01:** Все данные должны передаваться по протоколу HTTPS.
- **S01.02:** Хранение данных пользователя в зашифрованном виде.

4. Технические детали

- Язык разработки: Kotlin (Android), Swift (iOS).
- База данных: PostgreSQL.
- Интеграция с API смарт-часов (Apple Health, Google Fit).

5. Планы и этапы разработки

Этап №1: Анализ и проектирование — 1 месяц.

Этап №2: Разработка базового функционала — 2 месяца.

Этап №3: Тестирование — 1 месяц.

Этап №4: Запуск и поддержка — 1 месяц.

6. Требования к тестированию

1. **C06.01:** Все функции приложения должны быть протестированы на совместимость с реальными устройствами под управлением Android и iOS (последние три стабильные версии).
2. **R06.02:** Нагрузочное тестирование должно подтвердить корректную работу приложения при одновременном использовании 1000 активных пользователей.
3. **F06.03:** В рамках тестирования производительности необходимо убедиться, что время загрузки приложения не превышает 2 секунд при стандартной нагрузке.
4. **S06.04:** Безопасность приложения должна быть проверена на наличие уязвимостей
5. **U06.05:** Интерфейс пользователя должен быть протестирован на корректное отображение всех элементов в темной и светлой темах.

7. Ограничения и риски

- Возможные задержки при интеграции с API внешних устройств
- Ограничение бюджета на разработку
- Ограничения производительности мобильных устройств
- Риски, связанные с изменением требований или функционала

8. Критерии приемки

- Все функции работают согласно вышеуказанным требованиям.
- Отсутствуют критические ошибки.
- Приложение протестировано и доступно для скачивания в App Store и Google Play.

Ответы на контрольные вопросы

1. Что такое техническое задание?

Техническое задание или техзадание (ТЗ) — основной документ, содержащий требования заказчика к системе, в соответствии с которыми осуществляется создание и разработка конечного продукта. Такой документ описывает требования, цели, функциональные и нефункциональные характеристики будущего программного продукта.

ТЗ служит основой для планирования, разработки, тестирования и внедрения программного обеспечения, обеспечивая всем участникам проекта общее понимание того, что должно быть сделано.

2. Для кого составляется техническое задание?

Техническое задание составляется для разработчиков для понимания целей, задач и функциональных требований к системе; заказчиков, чтобы формализовать их ожидания и обозначить результат, который они получают; тестировщиков для определения критериев проверки и приемки продукта; менеджеров для контроля выполнения проекта и управления ресурсами; администраторов системы, чтобы понимать требования к эксплуатации.

3. Может ли техническое задание на разработку системы не содержать требований к тестированию, приемке? Если да, то в каком случае?

В исключительных случаях ТЗ может не содержать требований к тестированию и приемке, например, если система разрабатывается для личного использования либо ограничивается прототипом или экспериментальной системой, не предназначенной для ввода в эксплуатацию. Однако для большинства коммерческих и профессиональных проектов требования к тестированию и приемке являются обязательными.

4. Какие основные блоки входят в построение технического задания?

1. Цели и задачи проекта
2. Функциональные требования
3. Нефункциональные требования

4. Технические детали
5. Этапы и планы разработки
6. Требования к тестированию
7. Процесс внедрения
8. Ограничения и риски
9. Критерии приемки

5. Какие существуют этапы подготовительных работ по созданию технического задания?

1. Анализ требований
2. Исследование и анализ рынка
3. Определение целей и задач проекта
4. Разработка функциональных требований
5. Определение нефункциональных требований
6. Выбор технологий и инструментов
7. Разработка архитектуры системы
8. Планирование разработки
9. Разработка плана тестирования
10. Оценка рисков и ограничений
11. Документирование и согласование

6. Может ли техническое задание быть составлено не в соответствии с ГОСТ?

ТЗ может быть составлено не по ГОСТ если, например, используются внутренние стандарты компании или методологии или проект разработан для зарубежного рынка с ориентацией на международные стандарты.

7. Пример структуры технического задания по ГОСТ 34.602-2020

1. Общие сведения.
2. Назначение и цели разработки.
3. Характеристики объекта автоматизации.

4. Требования к системе:

- Функциональные.
- Нефункциональные.

5. Требования к документации.

6. Стадии и этапы разработки.

7. Порядок контроля и приемки.

8. Требования к поставке и монтажу.

9. Требования к гарантийному обслуживанию.

10. Приложения.

8. В чем основное структурное отличие ГОСТ 34.602-1989 и ГОСТ 34.602-2020?

ГОСТ 34.602-2020 уточняет требования к современным ИТ-системам, включая интеграции, кибербезопасность и документацию, а ГОСТ 34.602-1989 был больше ориентирован на автоматизированные системы управления (АСУ) и промышленные проекты.

9. Почему необходимо уметь составлять техническое задание?

- Для минимизации недоразумений между заказчиком и исполнителем.
- Для повышения качества разработки с помощью чёткого определения требований.
- Для обеспечения контроля над сроками и затратами.
- Для упрощения тестирования и приемки продукта.

10. Должны ли требования соответствовать определенным характеристикам? Если да, то каким?

Да, требования должны соответствовать характеристикам:

- **Необходимость.** Транслирует (описывает) реальную потребность стейкхолдеров.

- **Атомарность.** Не является механическим объединением разных требований.
- **Понятность.** Из описания всё понятно, нет необходимости обращаться за толкованием к заказчику.
- **Независимость от реализации.** Не накладывает преждевременные и ненужные технические ограничения.
- **Проверяемость.** Можно объективно проверить, выполнено ли требование.
- **Правдоподобность**

11. Для чего необходима идентификация требований в техническом задании?

Идентификация требований позволяет уникально идентифицировать каждое требование, упрощать ссылки на требования, исключать дублирование

12. Примеры систем идентификации требований:

1. Простая нумерация: 1, 2, 3...

Сильные стороны:

- Простота и понятность.
- Легкость в использовании и внедрении, особенно в небольших системах.
- Универсальность для простых проектов и задач.

Слабые стороны:

- Отсутствие связи между элементами и их функциями.
- Неудобство для больших систем: сложно добавить элементы, сохраняя логику, и поддерживать структуру.
- Непрозрачность: номера не несут смысловой нагрузки, поэтому сложно понять назначение элемента без дополнительного описания.

2. Иерархическая система: 1.1, 1.2, 2.1...

Сильные стороны:

- Удобство для структурированных и многоуровневых систем.

- Легко расширяется: можно добавлять новые элементы на любом уровне.
- Улучшает наглядность и понимание структуры системы.
- Хорошо подходит для систем с вложенностью, позволяя сохранять логику отношений.

Слабые стороны:

- Сложность при изменении структуры: добавление или удаление уровней требует корректировки всей системы нумерации.
- Могут возникать длинные иерархические обозначения, что снижает удобство восприятия.
- Сложнее для внедрения в небольших проектах без четкой иерархии.

3. **Мнемонические идентификаторы:** F01.02 (Функциональное, первый уровень)

Сильные стороны:

- Идентификаторы несут смысловую нагрузку, что облегчает их понимание и использование.
- Повышают наглядность и информативность системы.
- Удобство для технической документации и больших систем с функциональным разделением.
- Легкость поиска и навигации по элементам благодаря логичному структурированию.

Слабые стороны:

- Требуют четкого планирования и разработки системы идентификации.
- Более сложны в реализации по сравнению с простой нумерацией.
- Возможна избыточность и длина идентификаторов в больших системах.
- Потребность в обучении пользователей и поддержке единой системы стандартов.

Заключение

В ходе выполнения работы была достигнута поставленная цель — составлен текст технического задания для фитнес-приложения. Каждый пункт был подробно расписан в соответствии с требованиями. Были изучены методические материалы и интернет-ресурсы по заданной теме, что способствовало более глубокому пониманию структуры и содержания ТЗ. Кроме того, были проанализированы и даны развернутые ответы на контрольные вопросы. Составление ТЗ позволило приобрести практические навыки формулирования требований, систематизации информации и анализа

задач, что является важным этапом проектирования программного обеспечения. Выполненная работа подтвердила значимость разработки качественного ТЗ как основы для успешной реализации проекта.